

Prácticas de Ampliación de Sistemas Operativos •

minigrep

Descripción del programa minigrep

Escribe un programa llamado `minigrep` que lea un flujo de líneas (secuencias de bytes terminadas en `\n`) de la entrada estándar y muestre en la salida estándar aquellas líneas que contengan una subcadena reconocida por la expresión regular dada. Por ejemplo:

```
$ echo -e -n "1234\n5678\n90AB\n" | ./minigrep -r  
"^[[[:digit:]]67[[[:digit:]]$"  
5678  
$ echo -e -n "1234\n5678\n90AB\n" | ./minigrep -r  
"^[[[:digit:]]67[[[:digit:]]$" -v  
1234  
90AB  
$ ./minigrep -r "^[[[:digit:]]67[[[:digit:]]$"  
1234  
5678  
90AB  
5678
```

Ten en cuenta que en el último ejemplo, se introducen por teclado tres líneas con 1234, 5678 y 90AB respectivamente, pulsando tras el último retorno de carro `Ctrl+D` para indicar final de fichero. La última línea es la salida del programa, como se aprecia en el primer ejemplo.

El programa tiene que aceptar las siguientes opciones:

```
Uso: ./minigrep -r REGEX [-s BUFSIZE] [-v] [-h]  
-r REGEX Expresión regular.  
-s BUFSIZE Tamaño de los buffers de lectura y escritura en bytes (por  
defecto, 1024).  
-v Muestra las líneas que NO sean reconocidas por la expresión regular.
```

Consideraciones, indicaciones y requisitos

- Los argumentos de la línea de órdenes debe procesarse con `getopt()` y no podrá suponerse un orden concreto de dichos argumentos. Además podrá utilizarse más de una vez un argumento, en cuyo caso se tomará la última aparición del mismo como la correcta.
- `REGEX` es una expresión regular. En la [Referencia 1](#) encontrarás cómo compilar expresiones regulares y cómo reconocerlas en una línea de texto. En la [Referencia 2](#) tienes la especificación del formato de las expresiones regulares aceptadas. Se deberán utilizar los flags `REG_NEWLINE` y `REG_EXTENDED` en la llamada a `regcomp()`.
- `BUFSIZE` es el tamaño tanto del buffer de lectura como del buffer de escritura. Ambos buffers tendrán que reservarse en el *heap*. Tanto las lecturas como las escrituras *siempre* intentarán leer o escribir `BUFSIZE` bytes, es decir, leer y escribir la máxima cantidad de bytes que caben en el buffer. Las escrituras parciales deberán tratarse adecuadamente. El tamaño mínimo y máximo de los buffers será de 1 byte y 1 MB, respectivamente.
- Cuando se proporcione la opción `-v` se mostrarán las líneas que no sean reconocidas por la expresión regular en lugar de mostrar aquellas que sí lo sean.
- Para simplificar el problema, supondremos que **las líneas no tendrán más de 4096 bytes**, incluyendo el `\0` de fin de cadena. Si una línea fuera mayor, devolveríamos un error indicándolo e interrumpiríamos la ejecución del programa. Hay que tener en cuenta que para que funcione correctamente el reconocimiento del comienzo y final de línea en las expresiones regulares, no hay que incluir el `\n` final en la cadena. Por tanto, una línea será una secuencia de caracteres, hasta recibir un `\n` o detectar el final de fichero, seguida de un `\0`. El carácter `\n` no se incluirá en la cadena. Sin embargo, al imprimir la cadena en la salida estándar, siempre deberá escribirse un `\n` a continuación. Este es el comportamiento estándar de la orden `grep`. Por ejemplo:

```
$ echo -e -n "1234\n5678" | ./minigrep -r "^[[[:digit:]]23[[[:digit:]]$"
1234
$ echo -e -n "1234\n5678" | ./minigrep -r "^[[[:digit:]]67[[[:digit:]]$"
5678
$
```

- La manipulación de archivos debe realizarse con **llamadas al sistema** como, por ejemplo, `open()`, `read()`, `write()` o `close()`. En ningún caso deben usarse las funciones de la biblioteca estándar de C como, por ejemplo, `fopen()`, `fread()`, `fwrite()` o `fclose()`, y similares. Sin embargo, se permite usar `printf()` y `fprintf()` para mostrar mensajes de ayuda o de error.
- Se prohíbe el uso de funciones de manipulación de cadenas como, por ejemplo, `strcpy()` o `strcat()`, y similares.
- El programa `minigrep` lee de la entrada estándar, es decir, debe funcionar correctamente con un **archivo de entrada de tamaño infinito**. Por tanto, no se puede leer toda la entrada estándar, procesarla y escribir el resultado en la salida estándar. `minigrep` tiene que intercalar lecturas, procesamiento y escrituras hasta que agote la entrada.

Ejemplos de ejecución de minigrep

```
$ ./minigrep -h
Uso: ./minigrep -r REGEX [-s BUFSIZE] [-v] [-h]
    -r REGEX Expresión regular.
    -s BUFSIZE Tamaño de los buffers de lectura y escritura en bytes (por
defecto, 1024).
    -v Muestra las líneas que NO sean reconocidas por la expresión regular.

$ echo $?
0
$ ./minigrep -j
./minigrep: invalid option -- 'j'
Uso: ./minigrep -r REGEX [-s BUFSIZE] [-v] [-h]
    -r REGEX Expresión regular.
    -s BUFSIZE Tamaño de los buffers de lectura y escritura en bytes (por
defecto, 1024).
    -v Muestra las líneas que NO sean reconocidas por la expresión regular.
$ echo $?
1
$ echo -e -n "1234\n5678\n90AB\n" | ./minigrep -r 'ABCD' -r
"^[[[:digit:]]67[[[:digit:]]$"
5678
$ echo $?
0
```

Entrega y evaluación

- En la entrega se incluirá un únicamente el archivo `minigrep.c`.
- El archivo **deberá compilar** conforme a los archivos `tasks.json` o `Makefile` incluidos en `ASO-Semanal.tar.xz`, es decir, con `gcc -ggdb3 -Wall -Werror -Wno-unused -std=c11 minigrep.c -o minigrep`.
- La primera línea de `minigrep.c` deberá ser la directiva para compilar conforme al estándar POSIX: `#define_POSIX_C_SOURCE 200809L`.
- El fichero `minigrep.c` **deberá pasar todos los tests** proporcionados en `minigrep.json` para la utilidad de validación `asosystest.py`.

Para aprobar la práctica es imprescindible que el programa pase las pruebas de ejecución, sin que esto signifique que no se puedan detectar otros errores con pruebas adicionales que el profesor haga durante la evaluación.

Se tendrán en cuenta la correcta indentación del código, su estructuración, el manejo de los posibles errores de las llamadas al sistema o funciones de biblioteca, el uso de constantes en lugar de *números mágicos*, así como la correcta gestión de la memoria dinámica. Otro punto importante, y por eso lo volvemos a recordar, es que se penalizará severamente el no usar buffers para las lecturas/escrituras, es decir, leer o escribir byte a byte, pudiendo llegar a suspender la práctica, así como no contemplar lecturas o escrituras parciales si así se pide en el enunciado. Adicionalmente, las ineficiencias que se detecten en el código también podrán ser objeto de penalizaciones. Por ejemplo, reservar y liberar memoria dinámica o inicializar buffers innecesariamente.

Referencias

- [Linux Library Functions Manual: regex](#)
- [Linux Miscellaneous Information Manual: regex](#)